Introduction au Deep Learning

Un support de cours avec des exercices interactifs : entraîne un neurone, déplace une fonction d'activation et compare des optimizers directement dans la page.

À la fin, tu sauras :

- situer l'IA, le Machine Learning et le Deep Learning les uns par rapport aux autres ;
- expliquer le fonctionnement d'un neurone artificiel et d'un réseau multicouche ;
- choisir une fonction d'activation, un optimizer et un learning rate ;
- entraîner un réseau avec TensorFlow/Keras, sans fuite de données.

Un neurone qui apprend

Les portes logiques

Une porte logique est une petite règle de décision : elle reçoit deux réponses « oui » ou « non » (1 pour oui, 0 pour non) et rend une seule réponse, oui ou non. Le tableau indique la sortie pour chaque combinaison d'entrées (x_1, x_2).

Porte	La sortie vaut 1 quand…	(0, 0)	(0, 1)	(1, 0)	(1, 1)
AND (ET)	les deux entrées valent 1	0	0	0	1
OR (OU)	au moins une entrée vaut 1	0	1	1	1
NOR (NON-OU)	aucune entrée ne vaut 1 : c'est l'inverse de OR	1	0	0	0
XOR (OU exclusif)	une seule des deux entrées vaut 1	0	1	1	0

Un neurone seul apprend AND, OR et NOR : dans chaque cas, une droite suffit à séparer les sorties 0 des sorties 1. Pour XOR, aucune droite n'y arrive.

> **Il n'y a pas que ces quatre portes.** Avec deux entrées, il y a 4 cas possibles et chacun peut donner 0 ou 1 : 2 × 2 × 2 × 2 = 16 règles en tout. On en étudie quatre. Deux autres sont très courantes : NAND (l'inverse de AND) et XNOR (l'inverse de XOR). Sur les 16, 14 se séparent par une droite, donc un neurone seul les apprend ; seules XOR et XNOR demandent plusieurs neurones.

Un exemple à la main : NOR

Imagine une alarme qui reste calme (sortie 1) seulement si aucun des deux capteurs ne détecte quelque chose (x_1 et x_2 valent 1 quand le capteur détecte). Voici un neurone dont les valeurs sont déjà réglées : $\mathbf{w_1} = \mathbf{-2}$, $\mathbf{w_2} = \mathbf{-2}$ et $\mathbf{b} = \mathbf{+1}$. Il calcule `z = w₁·x₁ + w₂·x₂ + b`, puis répond 1 si z est positif, et 0 sinon. Change les capteurs et suis le calcul.

Capteur x_1 : **0** 1

Capteur x_2 : **0** 1

```
z = (−2 × x₁) + (−2 × x₂) + 1
z = (−2 × 0) + (−2 × 0) + 1 = 1
z est positif : la sortie est 1
```

x_1	x_2	z	Sortie du neurone	Sortie voulue (NOR)
0	**0**	**1**	**1**	**1**
0	1	−1	0	0
1	0	−1	0	0
1	1	−3	0	0

Capteurs (0, 0) : le neurone répond 1 et NOR attend 1 : c'est juste.

Pour l'expliquer simplement :

- Les poids négatifs (−2) font que chaque entrée à 1 **tire le total vers le bas**.
- Le biais positif (+1) donne au neurone un **point de départ favorable** : sans aucune entrée, il dit oui.
- Dès qu'une entrée vaut 1, le total passe sous zéro et le neurone dit non.

Et si le neurone ne connaît pas ces valeurs ?

Il les trouve en s'entraînant. Avec la sigmoïde, l'entropie croisée, $\eta = 0{,}5$ et 2000 époques :

- **Au départ**, $w = [0{,}50 ; -0{,}14]$ et $b = 0{,}65$: les sorties sont [0,66 ; 0,62 ; 0,76 ; 0,73]. Presque tout est « oui », le neurone ne sait encore rien.
- **À l'arrivée**, $w = [-8{,}66 ; -8{,}67]$ et $b = 3{,}87$: les sorties sont [0,98 ; 0,01 ; 0,01 ; 0,00], donc presque exactement 1, 0, 0, 0.

Le neurone retrouve le même schéma que l'exemple à la main : des poids négatifs et un biais positif. Les valeurs sont plus grandes parce que la sigmoïde ne donne une réponse nette que si z est loin de zéro.

Régler le laboratoire sur NOR

Le laboratoire

C'est l'exercice 1 du TP : un seul neurone apprend une porte logique par descente de gradient. Les poids de départ sont fixes (`seed 42`) : tu retrouves toujours les mêmes courbes. Choisis la porte, lance l'entraînement et regarde la frontière se déplacer.

Porte logique

AND | OR | NOR | XOR

Fonction de perte

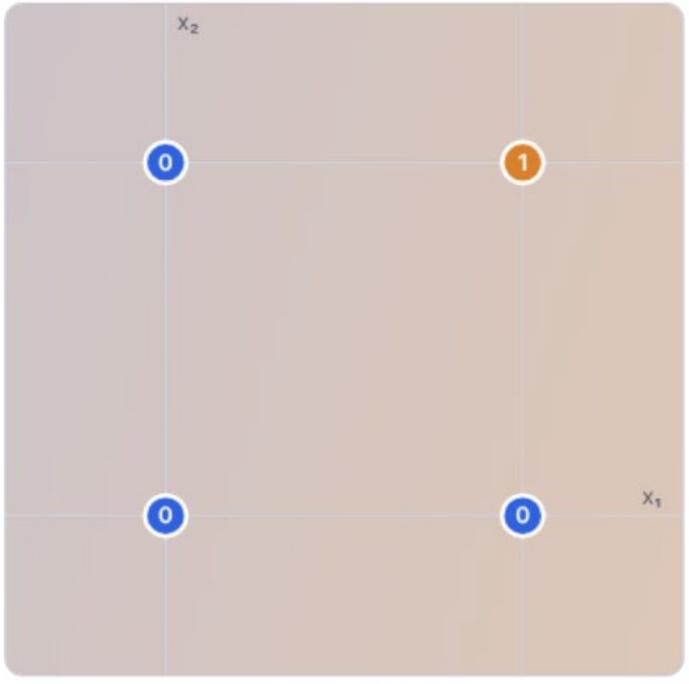
Erreur quadratique (MSE) | Entropie croisée

Learning rate η : **0,10**

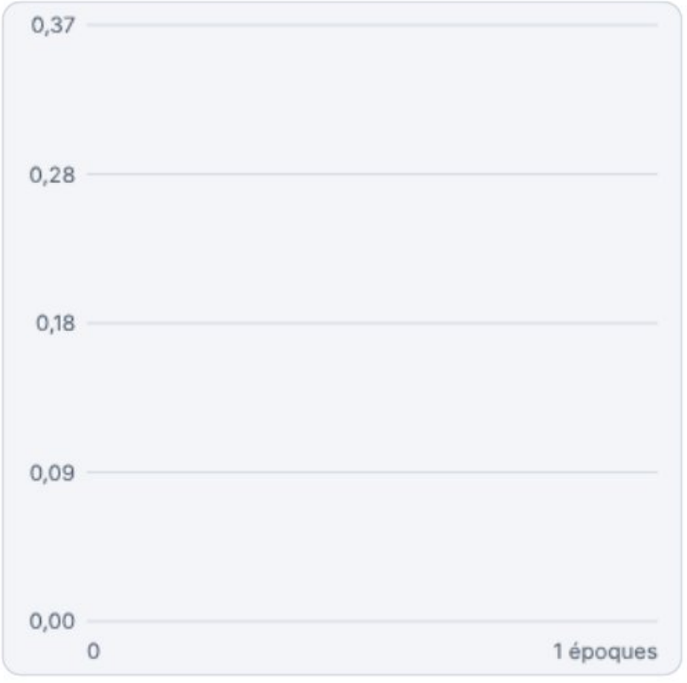
Époques par lancement

500 | 2000 | 5000

Entraîner | Une époque | Réinitialiser



Frontière de décision. Le fond montre la sortie du neurone, la ligne est le seuil 0,5.



Perte au fil des époques.

x_1	x_2	Cible	Sortie du neurone
0	0	0	0,6565
0	1	0	0,6247
1	0	0	0,7585
1	1	1	0,7323

Époque **0** Perte **0,3670** Exemples bien classés **1 / 4** Poids **w = [0,50 ; -0,14], b = 0,65**

Choisis une porte et lance l'entraînement. Les 4 points sont les 4 combinaisons d'entrées ; leur couleur est la cible.

À essayer. Avec la perte quadratique (MSE) et $\eta = 0{,}1$, la perte part de 0,367 et descend lentement : même après 2000 époques, les sorties restent loin de 0 et de 1. Passe à l'entropie croisée avec $\eta = 0{,}5$ et compare la vitesse. Termine par la porte NOR, puis par XOR pour voir la limite d'un neurone seul.

1 Introduction à l'IA, au Machine Learning et au Deep Learning

Définitions et hiérarchie

Intelligence artificielle

Toutes les techniques qui permettent à une machine de simuler l'intelligence humaine : résolution de problèmes, décision, compréhension du langage.

Machine Learning

La machine apprend à partir de données au lieu de suivre des règles écrites à la main.

Deep Learning

Réseaux de neurones à plusieurs couches (« deep » veut dire profond). Excellent sur les images, les sons, les textes.

Où on le croise

- **Voitures autonomes** : détection d'obstacles, lecture des panneaux, planification de trajectoire.
- **Assistants vocaux** : Siri, Alexa et Google Assistant comprennent la parole grâce au DL.
- **ChatGPT et les LLM** : modèles de langage fondés sur des réseaux profonds.
- **Reconnaissance faciale** : déverrouillage de téléphone, identification.
- **Recommandation** : Netflix, YouTube, Spotify personnalisent leurs suggestions.
- **Santé** : aide au diagnostic, analyse d'imagerie médicale.
- **Finance** : détection de fraude, trading algorithmique.

2 Machine Learning classique

Principe

Le ML cherche une fonction `f(x)` qui approxime la relation entre les entrées `x` (la surface d'une maison) et les sorties `y` (son prix), de sorte que `f(x) ≈ y` sur toutes les données.

Exemple : la régression linéaire

Le modèle le plus simple cherche la droite qui passe au plus près des points.

```
f(x) = a·x + b
```

`a` est la pente et `b` l'ordonnée à l'origine. Le modèle trouve les `a` et `b` qui rapprochent la droite de tous les points.

Fonction de coût

Elle mesure l'erreur du modèle : plus elle est grande, plus le modèle est mauvais. Pour la régression, on utilise souvent l'erreur quadratique moyenne :

```
L = (1/n) · Σ (y_réel − y_prédit)²
```

Descente de gradient

C'est l'algorithme qui minimise la fonction de coût en ajustant les paramètres petit à petit :

1. calculer l'erreur du modèle ;
2. calculer dans quelle direction modifier les paramètres pour la réduire (le gradient) ;
3. faire un petit pas dans cette direction ;
4. répéter jusqu'à convergence.

```
paramètre_nouveau = paramètre_ancien − learning_rate × gradient
```

Autres modèles classiques

- **Arbres de décision** : des règles « si condition A alors X, sinon Y ». Faciles à interpréter, mais ils sur-apprennent.
- **SVM** : cherchent l'hyperplan qui sépare au mieux les classes. Très efficaces en classification binaire.
- **KNN** : classe un point selon la classe majoritaire de ses k plus proches voisins.
- **Random Forest** : un ensemble d'arbres qui votent pour donner une prédiction plus robuste.

3 Histoire des réseaux de neurones

1943

McCulloch et Pitts. Premier modèle mathématique de neurone artificiel : des entrées sont combinées, et si la somme dépasse un seuil, le neurone s'active.

1957

Rosenblatt invente le perceptron. Premier algorithme capable d'apprendre automatiquement à partir de données, sur des problèmes linéairement séparables.

1969

Minsky et Papert. Ils démontrent qu'un perceptron simple ne résout pas les problèmes non linéaires comme XOR. Un désintérêt de plus de dix ans s'ensuit : le premier hiver de l'IA (1969-1980).

1986

Rétropropagation. Hinton et ses collaborateurs popularisent l'algorithme qui permet d'entraîner des réseaux multicouches (MLP). Le problème du XOR est résolu.

1989-1998

LeCun et les CNN. Réseaux convolutifs pour les images, dont LeNet-5, qui reconnaît les chiffres manuscrits. Leurs filtres détectent des motifs locaux (bords, textures) avant de les combiner.

Années 1990

RNN. Réseaux récurrents pour les séquences (texte, son, vidéo) : ils ont une « mémoire » du contexte.

2012

AlexNet. Un réseau de 8 couches gagne ImageNet avec une marge écrasante. Ses ingrédients : les GPU (jusqu'à 100 fois plus rapides), le Big Data (millions d'images), et ReLU, Dropout et la data augmentation.

2014-2015

VGG, Inception, ResNet. Des réseaux toujours plus profonds : ResNet atteint 152 couches grâce aux skip connections.

2017-2018

Transformers, BERT, GPT. Le mécanisme d'attention révolutionne le traitement du langage ; les modèles sont pré-entraînés.

2020 et après

GPT-3, DALL-E, ChatGPT. Des modèles géants à plusieurs milliards de paramètres.

Applications des CNN : reconnaissance d'images, détection d'objets, vision par ordinateur, imagerie médicale. Applications des RNN : traduction automatique, génération de texte, reconnaissance vocale, séries temporelles.

4 Le neurone artificiel et le perceptron

Anatomie d'un neurone

- **Entrées** $x_1 \dots x_n$: les données, ou les signaux venus d'autres neurones.
- **Poids** $w_1 \dots w_n$: l'importance de chaque entrée. Un poids élevé donne beaucoup d'influence.
- **Biais** b : une constante qui décale l'activation, comme un seuil personnalisé.
- **Somme pondérée** z, puis **fonction d'activation** f qui donne la sortie.

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b = \Sigma(w_ix_i) + b$$
$$y = f(z)$$

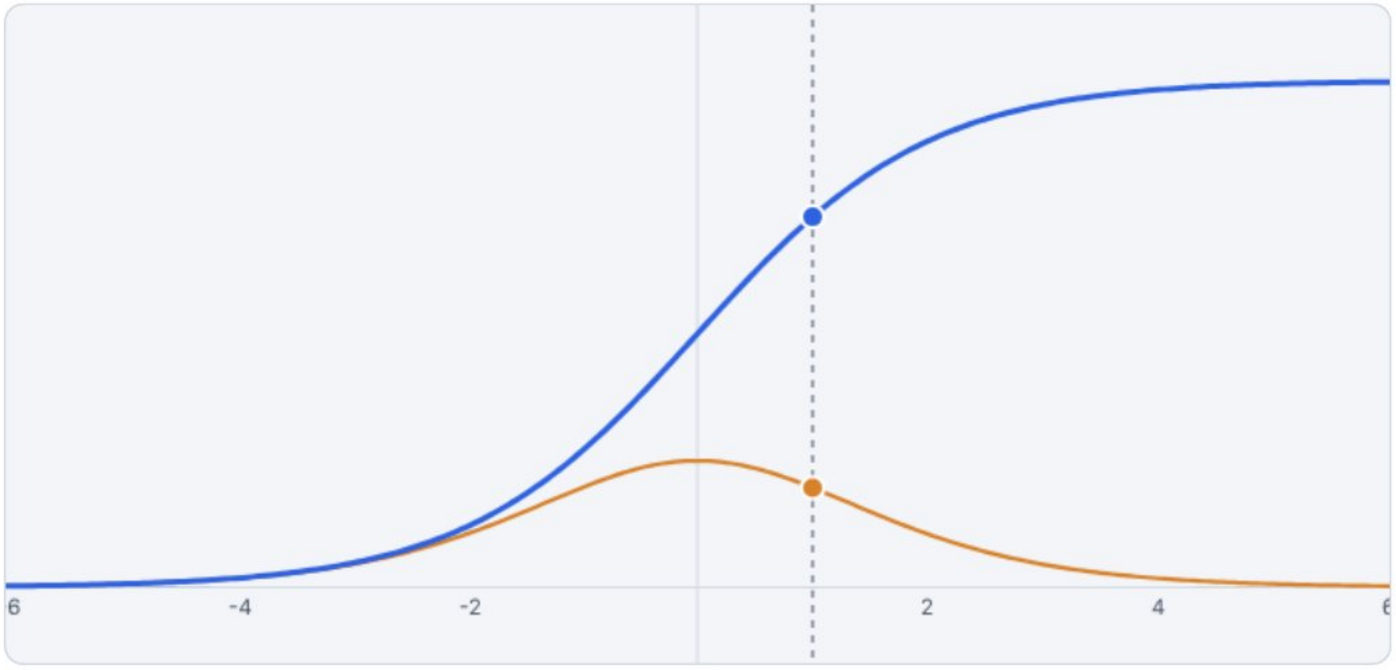
Les fonctions d'activation

Elles introduisent la non-linéarité, ce qui permet au réseau de modéliser des relations complexes. Choisis une fonction et déplace le curseur : la courbe bleue est `f(x)`, la courbe orange sa dérivée, c'est-à-dire ce qui est transmis au gradient.

Fonction

Sigmoïde | Tanh | ReLU | Leaky ReLU | GELU

Entrée x : **1,0**



f(x) en bleu, **f'(x)** en orange.

f(x) = **0,7311** f'(x) = **0,1966** Sortie dans **[0, 1]**

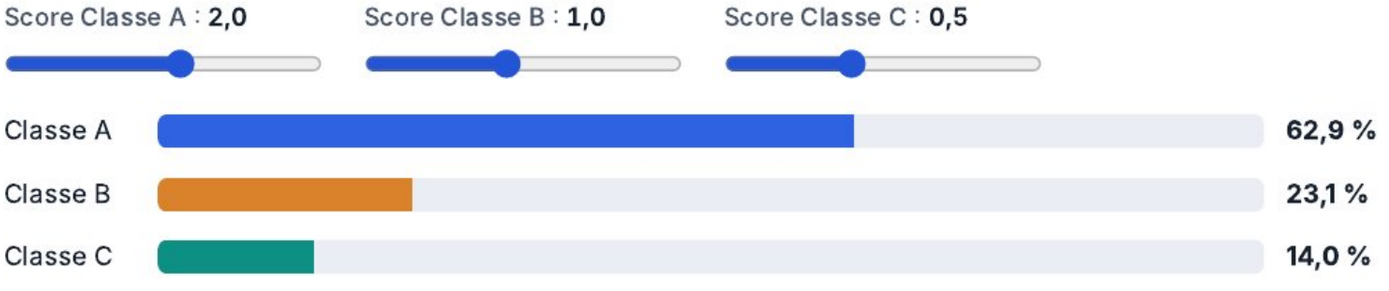
Plus x s'éloigne de 0, plus la dérivée s'écrase : essaie x = 5.

Fonction	Sortie	Problèmes principaux	Utilisation typique
Sigmoïde `1 / (1 + e⁻ˣ)`	[0, 1]	Vanishing gradient, non centrée en 0, exponentielle coûteuse	Sortie d'une classification binaire (probabilité de la classe positive)
Tanh `(eˣ − e⁻ˣ)/(eˣ + e⁻ˣ)`	[−1, 1]	Vanishing gradient aux extrêmes	Anciennes couches cachées, certains RNN et LSTM
ReLU `max(0, x)`	[0, ∞[	Dying ReLU : un neurone qui ne reçoit que des valeurs négatives reste bloqué à 0	Couches cachées (standard actuel) des CNN et des MLP
Softmax `eˣⁱ / Σⱼ eˣʲ`	[0, 1], somme = 1	Sensible aux valeurs extrêmes (overflow numérique)	Sortie d'une classification multi-classes, attention des Transformers

Variantes de ReLU : **Leaky ReLU** `max(0,01x, x)` garde un petit gradient pour $x < 0$; **PReLU** apprend ce coefficient ; **ELU** est lisse pour les valeurs négatives ; **GELU** sert dans les Transformers (GPT, BERT). Pour la tanh, `tanh(x) = 2·sigmoïde(2x) − 1` : c'est une sigmoïde remise à l'échelle.

Softmax : des scores aux probabilités

Softmax transforme des scores quelconques en probabilités qui somment à 1, et accentue les écarts : le score le plus haut prend une part bien plus grande.



Exemple. Pour les scores [2,0 ; 1,0 ; 0,5], softmax donne [0,63 ; 0,23 ; 0,14] : la première classe a 63 % de probabilité d'être la bonne. Vérifie avec les curseurs ci-dessus.

Le perceptron simple

C'est le plus petit réseau possible : un seul neurone, avec n entrées, n poids, un biais, une fonction d'activation (en général une fonction échelon ou la sigmoïde) et une sortie. On calcule $z = \Sigma(w_i x_i) + w_0$, puis $y = f(z)$; si y dépasse un seuil, c'est la classe 1, sinon la classe 0.

Règle d'apprentissage

Pour chaque exemple (x, y_vrai) :

1. $y_prédit = f(\Sigma\ w_i x_i + w_0)$
2. $erreur\ e = y_vrai - y_prédit$
3. $w_i \leftarrow w_i + \eta \cdot e \cdot x_i$
4. $w_0 \leftarrow w_0 + \eta \cdot e$

η (êta) est le learning rate.

> **Une nuance de vocabulaire.** La règle ci-dessus (activation en échelon, mise à jour par l'erreur) est celle du perceptron de Rosenblatt. L'exercice 1 fait autre chose : une sigmoïde, une perte quadratique et un vrai gradient. C'est un neurone logistique entraîné par descente de gradient, ce qui prépare directement la rétropropagation du module 5. Les deux se limitent aux problèmes linéairement séparables.

Limites

- Il ne résout que les problèmes **linéairement séparables**.
- Il ne peut pas apprendre **XOR** (essaie dans le laboratoire du début).
- Sa puissance de représentation est limitée.

La solution : empiler plusieurs couches de neurones.

5 Réseaux de neurones multicouches (MLP)

Architecture

- **Couche d'entrée** : reçoit les données brutes. Autant de neurones que de features, aucun calcul.
- **Couches cachées** : transforment les données et extraient des features. Il peut y en avoir plusieurs (d'où « deep »). Chaque neurone est relié à tous ceux de la couche précédente (*fully connected*).
- **Couche de sortie** : produit la prédiction. 1 neurone pour une régression ou une classification binaire, k neurones pour k classes.

Forward propagation

Calculer la sortie du réseau à partir des entrées, couche après couche : `z[l] = W[l]·a[l−1] + b[l]`, puis `a[l] = f(z[l])`. La sortie de la dernière couche est la prédiction ŷ.

```
Entrée : x = [x₁, x₂]
Couche cachée 1 : z₁ = W₁·x  + b₁   a₁ = ReLU(z₁)
Couche cachée 2 : z₂ = W₂·a₁ + b₂   a₂ = ReLU(z₂)
Sortie :          z₃ = W₃·a₂ + b₃   ŷ  = sigmoïde(z₃)
```

Rétropropagation

Elle calcule le gradient de la perte par rapport à chaque poids et chaque biais :

1. calculer l'erreur en sortie, par exemple `δ_sortie = (ŷ − y) · f′(z_sortie)` ;
2. la propager vers les couches précédentes avec la règle de la chaîne ;
3. pour chaque couche, calculer la contribution de chaque poids à l'erreur finale.

Mise à jour des paramètres

```
W[l] ← W[l] − η · ∂L/∂W[l]
b[l] ← b[l] − η · ∂L/∂b[l]
```

La boucle d'entraînement complète

```
Pseudo-code

pour chaque epoch (un passage sur toutes les données) :
    pour chaque batch :
        1. forward propagation  → ŷ
        2. calcul de la loss    L(ŷ, y)
        3. backpropagation      → tous les gradients
        4. mise à jour des paramètres avec l'optimizer
    évaluer la performance sur le jeu de validation
```

Pourquoi les réseaux profonds fonctionnent

Ils construisent une hiérarchie de représentations. Pour une image : couche 1, des bords ; couche 2, des formes simples (cercles, rectangles) ; couche 3, des parties d'objets (yeux, roues, fenêtres) ; couche 4, des objets complets (visage, voiture, maison).

Le théorème d'approximation universelle dit qu'un réseau avec au moins une couche cachée peut approximer n'importe quelle fonction continue. En pratique, les réseaux profonds l'apprennent plus efficacement.

6 Les optimizers

Un optimizer ajuste les poids et les biais pour minimiser la fonction de coût. Le principe reste la descente de gradient `w ← w − η · ∂L/∂w`. Imagine que tu descends une montagne les yeux bandés : le gradient indique la plus forte pente, l'optimizer décide comment et à quelle vitesse descendre.

La course des optimizers

Les quatre optimizers partent du même point d'une vallée étroite, $f(x, y) = \frac{1}{2}(x^2 + 20y^2)$, et visent le minimum au centre. Fais varier le learning rate : c'est aussi la démonstration du module 7.

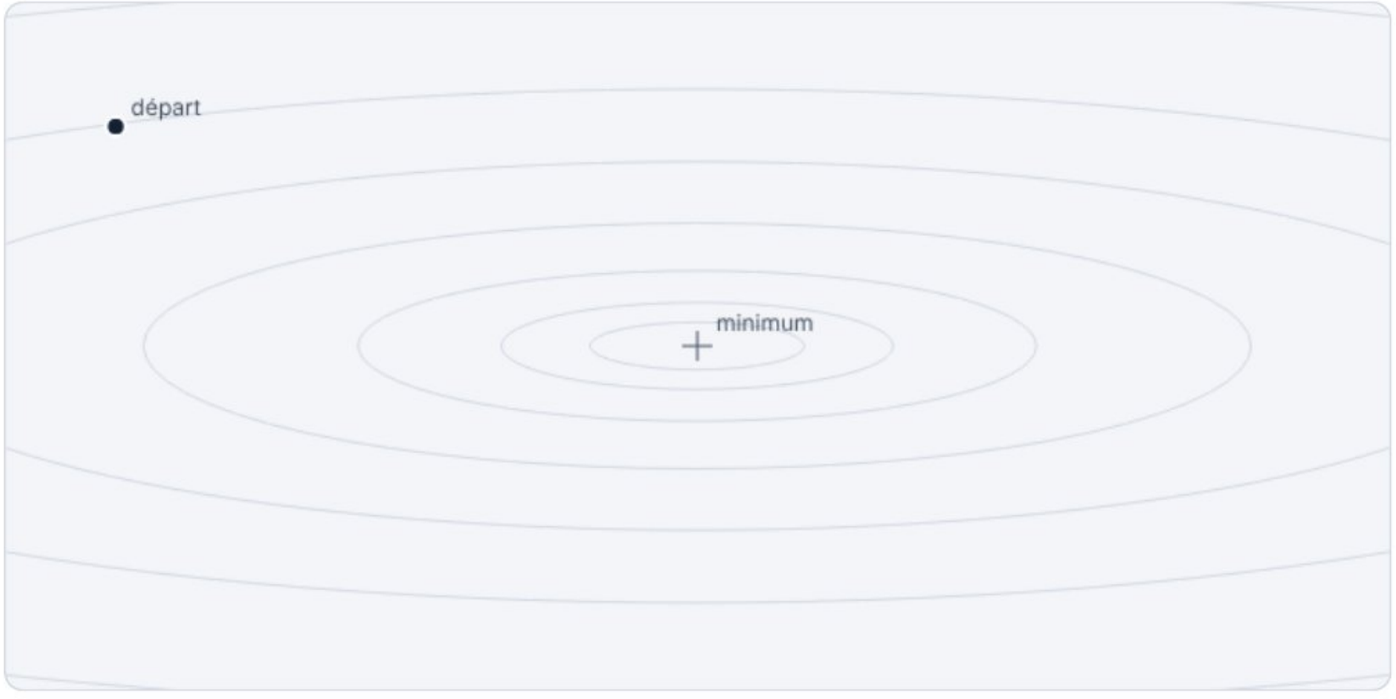
Learning rate η : **0,05**

Raccourcis

0,01 trop petit | **0,05** | 0,12 SGD diverge

☐ Step decay (η ÷ 2 tous les 50 pas)

Lancer | Réinitialiser



SGD : **f = 34,420** Momentum : **f = 34,420** RMSProp : **f = 34,420** Adam : **f = 34,420**

Pas **0** / 300 η courant **0,050**

SGD (descente de gradient stochastique)

Met à jour les poids après chaque mini-batch : `w ← w − η · ∂L/∂w`. Très simple, peu de mémoire, excellent pour comprendre. En revanche la convergence est lente, la trajectoire bruitée, très sensible au learning rate, et les données d'échelles différentes lui posent problème. Usages : apprentissage, petits modèles, cas simples.

Momentum

Ajoute de l'inertie : les poids gardent une partie de leur mouvement précédent, comme une balle qui roule et passe les petits creux. Il converge plus vite et oscille moins, mais ajoute un hyperparamètre β (en général 0,9) et peut dépasser le minimum si le momentum est trop élevé. Variante : Nesterov, qui « regarde en avant » avant de calculer le gradient.

```
v ← β·v + ∂L/∂w        (vitesse accumulée)
w ← w − η·v            (mise à jour)
```

RMSProp

Adapte le pas de chaque paramètre à l'historique de ses gradients : un paramètre aux gros gradients reçoit un pas plus petit, et inversement. Il gère bien les échelles différentes et les données séquentielles (RNN, LSTM), mais peut être instable. Il a été développé par Hinton dans son cours Coursera.

```
s ← β·s + (1 − β)·(∂L/∂w)²          β ≈ 0,9
w ← w − η · (∂L/∂w) / (√s + ε)      ε = 10⁻⁸
```

Adam

Combine Momentum (moyenne mobile des gradients) et RMSProp (moyenne mobile de leurs carrés), avec une correction de biais pour les premières itérations. C'est le standard du Deep Learning, robuste « out of the box ». Il consomme plus de mémoire que SGD (m et v par paramètre), peut sur-ajuster sur de petits jeux de données et converge parfois vers des minima moins généralisables que SGD. Variantes : AdamW (meilleure régularisation par weight decay), Nadam, RAdam.

```
m ← β₁·m + (1 − β₁)·∂L/∂w
v ← β₂·v + (1 − β₂)·(∂L/∂w)²
m̂ = m / (1 − β₁ᵗ)        v̂ = v / (1 − β₂ᵗ)
w ← w − η · m̂ / (√v̂ + ε)

défauts : β₁ = 0,9   β₂ = 0,999   η = 0,001   ε = 10⁻⁸
```

Optimizer	Principe	Avantages	Inconvénients	Usages courants
SGD	Descente de gradient par batch	Simple, pédagogique	Lent, sensible au LR	Cas simples, apprentissage
Momentum	Ajoute une inertie	Plus stable, évite la stagnation	Un paramètre de plus	Réseaux de base
RMSProp	LR adapté par paramètre	Bonne convergence	Parfois instable	RNN, séquences
Adam	Momentum + RMSProp	Rapide, efficace, robuste	Peut sur-ajuster	Standard actuel

Comment choisir

- Pour débuter : Adam avec ses paramètres par défaut ($\eta = 0{,}001$).
- En production : Adam reste le meilleur choix dans la grande majorité des cas.
- En cas de sur-apprentissage : essaie SGD avec Momentum, ou AdamW.
- Pour un RNN ou un LSTM : RMSProp ou Adam.
- Pour comprendre : commence par implémenter SGD toi-même.

7 Le learning rate

Le learning rate η est l'hyperparamètre qui contrôle la taille du pas effectué lors de la mise à jour : `w ← w − η · ∂L/∂w`, où le gradient donne la direction et η l'amplitude. C'est le paramètre le plus important à régler, quel que soit l'optimizer.

Intuition

- **η petit (0,0001)** : descente lente mais stable, qui ne dépasse pas la vallée. L'entraînement prend énormément de temps et peut rester coincé dans un petit creux.
- **η grand (0,1)** : descente rapide au début, mais le risque est de sauter par-dessus la vallée. Le modèle peut diverger : la loss explose.

- **Bon η (0,001 à 0,01)** : équilibre entre rapidité et stabilité, avec des ajustements expérimentaux.

Reprends la course des optimizers du module 6 et monte η à 0,12 : SGD diverge, alors qu'Adam et Momentum tiennent encore.

Optimizer et learning rate : deux rôles

Concept	Rôle	Analogie
Optimizer	Définit la direction de la mise à jour (comment naviguer)	Le moteur
Learning rate	Définit l'amplitude de cette mise à jour (à quelle vitesse)	La vitesse du véhicule

```Keras
optimizer = SGD(learning_rate=0.01)     # dépend totalement d'un bon η
optimizer = Adam(learning_rate=0.001)   # adapte les pas, mais reste sensible à η
```

Valeurs de départ

Optimizer	η recommandé	Plage courante
SGD	0,01 à 0,1	0,001 à 1,0
SGD + Momentum	0,01 à 0,1	0,001 à 0,1
RMSProp	0,001	0,0001 à 0,01
Adam	0,001 (défaut)	0,0001 à 0,01

Stratégies pour trouver le bon η

1. **Les valeurs par défaut** : 0,001 pour Adam, 0,01 pour SGD.
2. **Le Learning Rate Finder** (Leslie Smith) : augmenter progressivement η pendant l'entraînement et noter quand la loss se met à diverger. Le meilleur η se situe juste avant que la loss remonte.
3. **Start high, go low** : partir d'un η assez élevé (0,01) ; si la loss oscille, le diviser par 10 ; si elle baisse trop lentement, le multiplier par 2 ou 3.

Learning rate scheduling

Plutôt qu'un η fixe, on le fait varier pendant l'entraînement. Le **step decay** le réduit par paliers à intervalles fixes : simple, efficace, très utilisé. Coche « Step decay » dans la course des optimizers pour voir l'effet.

Symptômes et solutions

Symptôme	Cause probable	Solution
La loss diminue très lentement	η trop petit	Augmenter η (×2 ou ×10)
La loss oscille fortement	η trop grand	Réduire η (÷2 ou ÷10)
La loss explose (NaN)	η beaucoup trop grand	Réduire drastiquement η (÷100)
La loss stagne après un moment	η devenu trop grand	Utiliser un schedule (decay)
Convergence rapide, puis stagnation	Bon η initial, mais il ne s'adapte pas	Learning rate scheduling

Travaux pratiques

Cinq exercices, du neurone isolé au réseau récurrent. Le code ne s'exécute pas dans la page : recopie-le à la main dans ton notebook, c'est la meilleure façon de l'apprendre.

Exercice 1 : le perceptron

Objectif : résoudre une classification binaire (la porte logique AND) avec un neurone et la descente de gradient. Tu l'as déjà fait en direct tout en haut de la page ; voici le code NumPy. Il utilise l'entropie croisée, dont le gradient se simplifie en `ŷ − y`.

Python, NumPy

```python
import numpy as np

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1])                  # AND ; pour NOR : np.array([1, 0, 0, 0])

np.random.seed(42)
w = np.random.randn(2)
b = np.random.randn()

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def predict(X, w, b):
    return sigmoid(X @ w + b)

def bce(y, p, eps=1e-12):                   # entropie croisée binaire
    return -np.mean(y * np.log(p + eps) + (1 - y) * np.log(1 - p + eps))

lr, epochs = 0.5, 2000
for i in range(epochs):
    p = predict(X, w, b)
    dz = p - y                              # gradient de l'entropie croisée
    w -= lr * (X.T @ dz) / len(X)
    b -= lr * dz.mean()
    if i % 200 == 0:
        print(f"Epoch {i}, Loss: {bce(y, p):.4f}")

print("Prédictions :", predict(X, w, b).round(3))
```

À toi : refais l'exercice avec la porte NOR.

Exercice 2 : Iris avec un MLP

Objectif : classer les fleurs Iris en 3 classes à partir de 4 mesures. Retiens l'ordre des opérations : on **sépare d'abord**, puis on ajuste le `StandardScaler` sur le train seulement. Sinon la moyenne et l'écart-type du jeu de test influencent l'entraînement (fuite de données).

Python, Keras

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Dense

X, y = load_iris(return_X_y=True)           # 4 features, classes 0, 1, 2

# 1. on sépare d'abord
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 2. on ajuste le scaler sur le train uniquement
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)

model = Sequential([
    Input(shape=(4,)),
    Dense(8, activation="relu"),
    Dense(3, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",   # y_train contient 0, 1, 2 (pas de one
              metrics=["accuracy"])
model.summary()

history = model.fit(X_train, y_train, epochs=300, batch_size=16,
                    validation_split=0.1, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

Le test donne environ 90 % de bonnes réponses. Avec 100 époques seulement, le réseau n'a pas fini d'apprendre : regarde les courbes `history.history` pour le constater.

Exercice 2, suite : Titanic

Objectif : prédire la survie d'un passager (classification binaire). Étapes, dans l'ordre :

1. Choisir les colonnes pertinentes (`features`).

2. Séparer les données en train et test.
3. Gérer les valeurs manquantes, avec les statistiques du train.
4. Encoder les colonnes catégorielles.
5. Normaliser les colonnes numériques.
6. Convertir en tableaux numpy.
7. Vérifier les dimensions.

Toutes les statistiques (médiane de l'âge, mode de l'embarquement, moyenne et écart-type du scaler) se calculent sur le train. `stratify` garde la même proportion de survivants dans les deux jeux, et l'arrêt anticipé interrompt l'entraînement quand la validation ne progresse plus.

Python, pandas et Keras

```
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
from tensorflow.keras import Sequential, Input
from tensorflow.keras.layers import Dense
from tensorflow.keras.callbacks import EarlyStopping

titanic = sns.load_dataset("titanic")
features = ["pclass", "sex", "age", "sibsp", "parch", "fare", "embarked"]
X, y = titanic[features], titanic["survived"].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 1. valeurs manquantes : on utilise les statistiques du train
fill = {"age": X_train["age"].median(), "embarked": X_train["embarked"].mode()[0]}
X_train = X_train.fillna(fill)
X_test = X_test.fillna(fill)

# 2. variables catégorielles (mêmes colonnes pour train et test)
cat = ["sex", "embarked", "pclass"]
X_train = pd.get_dummies(X_train, columns=cat, drop_first=True)
X_test = pd.get_dummies(X_test, columns=cat, drop_first=True)
X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

# 3. normalisation des colonnes numériques (fit sur le train)
num = ["age", "fare"]
scaler = StandardScaler().fit(X_train[num])
X_train[num] = scaler.transform(X_train[num])
X_test[num] = scaler.transform(X_test[num])

# 4. conversion en tableaux numpy et vérification des dimensions
X_train = X_train.values.astype("float32")
X_test = X_test.values.astype("float32")
print("Shape X_train:", X_train.shape, "| Shape X_test:", X_test.shape)

model = Sequential([
    Input(shape=(X_train.shape[1],)),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid"),
])
```

```
model.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])

early = EarlyStopping(patience=10, restore_best_weights=True)
history = model.fit(X_train, y_train, epochs=100, batch_size=32,
                    validation_split=0.1, callbacks=[early], verbose=0)

pred = (model.predict(X_test) > 0.5).astype(int).ravel()
print(classification_report(y_test, pred))
```

Regarde ensuite `history.history` : si la perte de validation remonte alors que celle d'entraînement continue de baisser, c'est du sur-apprentissage. Compare 16 neurones cachés avec 4 puis avec 64.

Exercice 3 : un CNN sur MNIST

Objectif : classer des images de chiffres manuscrits (0 à 9) avec un réseau convolutif. Le réseau compte 130 890 paramètres, et la dernière couche convolutive sort 3 × 3 × 64 = 576 valeurs, que `Flatten` met à plat. La validation est prélevée dans le train, pour que le test reste inconnu jusqu'à l'évaluation finale.

Python, Keras

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras import datasets, layers, models, Input
from sklearn.metrics import confusion_matrix, classification_report

(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()

# une image brute (valeurs 0 à 255), avant toute transformation
img = X_train[y_train == 5][0]
plt.imshow(img, cmap="gray")
plt.title("Original (0-255)")
plt.axis("off")
plt.show()

# normalisation (0 à 1) puis ajout du canal : (N, 28, 28) -> (N, 28, 28, 1)
X_train = np.expand_dims(X_train.astype("float32") / 255.0, -1)
X_test = np.expand_dims(X_test.astype("float32") / 255.0, -1)
print("Formes finales :", X_train.shape, X_test.shape)

model = models.Sequential([
    Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.Flatten(),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.5),
    layers.Dense(10, activation="softmax"),
])
model.compile(optimizer="adam",
              loss="sparse_categorical_crossentropy",
              metrics=["accuracy"])
model.summary()

# la validation est prise dans le train : le test reste inconnu jusqu'à l'évaluation finale
history = model.fit(X_train, y_train, epochs=5, batch_size=64, validation_split=0.1)

loss, acc = model.evaluate(X_test, y_test, verbose=2)
print(f"\nPrécision sur le test : {acc * 100:.2f}%")
```

```
y_pred = np.argmax(model.predict(X_test), axis=1)

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues")
plt.xlabel("Prédit")
plt.ylabel("Réel")
plt.show()

print(classification_report(y_test, y_pred))

plt.imshow(X_test[0].squeeze(), cmap="gray")
plt.title(f"Prédit : {y_pred[0]}, Réel : {y_test[0]}")
plt.show()
```

À faire :

- Prépare une présentation qui explique le fonctionnement d'un CNN et le rôle de chaque couche : `Conv2D`, `MaxPooling2D`, `Flatten`, `Dense`, `Dropout`.
- Explique le code ci-dessus étape par étape : prétraitement, architecture, entraînement, évaluation.

Exercice 4 : un CNN sur Fashion-MNIST

Objectif : classer des images de vêtements en 10 catégories. Les données ont le même format que MNIST (images de 28 × 28 pixels, 60 000 pour l'entraînement et 10 000 pour le test).

Python, Keras

```
import tensorflow as tf

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()
print(x_train.shape, y_train.shape)        # (60000, 28, 28) (60000,)

classes = ["T-shirt/top", "Pantalon", "Pull", "Robe", "Manteau",
           "Sandale", "Chemise", "Sneaker", "Sac", "Bottine"]
```

À toi : reprends les étapes de l'exercice 3 (normalisation, canal, CNN, entraînement, évaluation, matrice de confusion) et repère les classes que le réseau confond le plus.

Exercice 5 : un RNN qui génère du texte

Objectif : prédire le mot suivant à partir des 3 mots précédents, puis écrire un texte mot après mot. Prépare un fichier `textes.txt` (UTF-8) avec un texte par ligne. Les exemples d'entraînement et la génération utilisent le même format : une fenêtre de 3 mots, complétée par des zéros au début si besoin.

Python, Keras

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.callbacks import EarlyStopping

with open("textes.txt", "r", encoding="utf-8") as f:
    texts = [line.strip() for line in f if line.strip()]
print(f"Exemple de texte : {texts[0]}")

vocab_size = 1000
window_size = 3

tokenizer = Tokenizer(num_words=vocab_size)
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)

# exemples d'apprentissage : au plus 3 mots précédents -> mot suivant
X, y = [], []
for seq in sequences:
    for i in range(1, len(seq)):
        X.append(seq[max(0, i - window_size):i])
        y.append(seq[i])
X = pad_sequences(X, maxlen=window_size, padding="pre")
y = np.array(y)
print(f"X shape: {X.shape}, y shape: {y.shape}")

embedding_dim = 16
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=embedding_dim),
    SimpleRNN(32),
    Dense(vocab_size, activation="softmax"),
])
model.compile(loss="sparse_categorical_crossentropy", optimizer="adam", metrics=["accuracy"])

early_stop = EarlyStopping(monitor="loss", patience=10, restore_best_weights=True)
history = model.fit(X, y, epochs=500, batch_size=32, callbacks=[early_stop])

plt.plot(history.history["loss"])
plt.title("Loss pendant l'entraînement")
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
plt.show()

def generate_text(start_text, num_words=50, temperature=0.8):
    text = start_text
    for _ in range(num_words):
        # même format qu'à l'entraînement : les 3 derniers mots seulement
        tokens = tokenizer.texts_to_sequences([text])[0][-window_size:]
        tokens = pad_sequences([tokens], maxlen=window_size, padding="pre")
        probs = model.predict(tokens, verbose=0)[0]

        # tirage au sort pondéré (temperature < 1 : plus sûr, > 1 : plus varié)
        probs = np.exp(np.log(probs + 1e-9) / temperature)
        probs /= probs.sum()
        index = np.random.choice(len(probs), p=probs)

        word = tokenizer.index_word.get(index)
        if word is not None:                # l'indice 0 sert au bourrage, pas un mot
            text += " " + word
    return text

print(generate_text("le poète", num_words=50))
```

Teste ensuite un LSTM : remplace `SimpleRNN(32)` par `LSTM(64)` (import : `from tensorflow.keras.layers import LSTM`) et compare la loss et le texte obtenu. Change aussi `temperature` : plus elle est basse, plus le texte se répète.

Questions

1. Que fait `Tokenizer` et pourquoi est-il nécessaire avant un RNN ?
2. Pourquoi utilise-t-on `pad_sequences`, et que se passerait-il sans lui ?
3. Quelle est la différence entre `vocab_size` et `embedding_dim` ?
4. En quoi consiste une couche `Embedding`, et pourquoi la place-t-on avant le RNN ?
5. Quelle est la différence entre `SimpleRNN` et `LSTM` ? Dans quel cas préférer un LSTM ?
6. Que représente `window_size = 3` et comment influence-t-il l'apprentissage ?
7. Pourquoi utiliser `sparse_categorical_crossentropy` dans ce cas précis ?
8. Que fait le callback `EarlyStopping` et pourquoi est-il utile ici ?

9. En observant la courbe de la loss, comment savoir si le modèle converge bien ?
10. Analyse le texte généré : est-il grammaticalement correct et cohérent ? Pourquoi le modèle n'écrit-il pas encore comme un auteur ?

Auto-évaluation

Six questions pour vérifier l'essentiel.

Quelle fonction d'activation place-t-on sur la couche de sortie pour reconnaître 10 chiffres (MNIST) ?

- Sigmoïde
- ReLU
- Softmax
- Tanh

Pourquoi un neurone seul ne peut-il pas apprendre XOR ?

- Il manque de données
- XOR n'est pas linéairement séparable
- Le learning rate est trop petit
- La sigmoïde sature

Pour x > 0, quel est le gradient de ReLU ?

- 0
- Il tend vers 0
- 0,01
- 1

La loss devient NaN en cours d'entraînement. Première piste ?

- Learning rate beaucoup trop grand
- Learning rate trop petit
- Trop peu de couches
- Pas assez d'époques

Adam combine quels deux mécanismes ?

- SGD et dropout
- Momentum et RMSProp
- Step decay et Nesterov
- Softmax et ReLU

Pourquoi ajuster le StandardScaler sur le jeu d'entraînement seulement ?

- C'est plus rapide
- Pour éviter que des informations du jeu de test fuient dans le modèle
- Keras l'exige
- Pour garder des valeurs entre 0 et 1

Support de cours « Introduction au Deep Learning », 2025-2026. Le contenu de cette page est protégé : la copie est désactivée.